

# SMP Construction

How the kernel, the ports and the boards fit together

# SMP construction

How the  $\mu$ OS++ III SMP kernel, an architecture port, the device drivers and the test applications fit together, and what each one owes the others.

## 1. The six repositories

|                              |                                                      |                    |
|------------------------------|------------------------------------------------------|--------------------|
| micro-os-plus-iii-smp        | the kernel, the shared tests, the shared build rules |                    |
| micro-os-plus-iii-devices    | SD, flatfs, DWC2, FatFs, BCM2837 SoC support         |                    |
| micro-os-plus-iii-aarch32    | ARMv7-A port                                         |                    |
| micro-os-plus-iii-aarch64    | ARMv8-A port                                         |                    |
| micro-os-plus-iii-cortexm    | ARMv7E-M / ARMv8-M port                              | (not yet migrated) |
| micro-os-plus-iii-posix-arch | native host port                                     | (not yet migrated) |

Dependencies run **one way**. An architecture project consumes the kernel and, if it needs them, the devices; neither of those knows an architecture project exists. Nothing is vendored: the projects are cloned side by side and CMake finds them as siblings.



Keeping the kernel tree free of device and board code is what lets `src/` and `include/` match upstream micro-os-plus-iii path for path, so the fork can be merged against upstream later with conflicts only in the files that genuinely differ.

## 2. Division of labour

| Concern                                                      | Lives in                                                    | Why there                                                               |
|--------------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------|
| Scheduler, threads, sync objects, per-CPU run state          | kernel <code>src/rtos/</code>                               | no machine instructions; identical for every port                       |
| Type widths, IRQ state representation, core count            | port <code>include/cmsis-plus/rtos/port/os-c-decls.h</code> | the only place a word size is decided                                   |
| Instructions: barriers, IRQ mask, core id, exclusive monitor | port <code>include/cmsis-plus/rtos/port/os-inlines.h</code> | one file per ISA                                                        |
| Bring-up, kernel lock, context switch hooks                  | port <code>src/rtos/os-core.cpp</code>                      | the port's half of the scheduler                                        |
| Reset vector, MMU, exception vectors, timer                  | port <code>src/</code>                                      | board and ISA                                                           |
| SD, FatFs, flatfs, USB, SoC mailbox                          | devices repo                                                | shared verbatim by both ARM ports                                       |
| Test applications                                            | port <code>test/&lt;board&gt;/</code>                       | every board owns its own — see <code>tests-in-aarch32-aarch64.md</code> |

### Two files called `src/rtos/os-core.cpp`

This is deliberate and both are compiled into every image.

- The **kernel's** is the portable SMP scheduler: per-CPU `current_thread_[]`, the affinity picker, the per-core idle bookkeeping. It contains no machine instructions and calls out to `port_cpu_id()`.
- The **port's** is the bring-up and the lock: secondary core release, the recursive kernel spinlock, the context-switch entry points.

They share a path because each is at the path its own project expects; they do not collide, and the predecessor Makefiles compiled both in exactly the same way.

## 3. The SMP port contract

The whole kernel-side SMP patch is 379 lines and asks a port for a small, closed set of things.

### 3.1 Types — `cmsis-plus/rtos/port/os-c-decls.h`

```
typedef uint64_t os_port_clock_timestamp_t;
typedef uint32_t os_port_clock_duration_t;
typedef uint64_t os_port_clock_offset_t;
typedef bool     os_port_scheduler_state_t;
typedef uint64_t os_port_irq_state_t;          /* AArch32: uint32_t */
typedef uint64_t os_port_thread_stack_element_t; /* AArch32: uint32_t */
typedef uint64_t os_port_thread_stack_allocation_element_t;

typedef struct { os_port_thread_stack_element_t* stack_ptr; }
                os_port_thread_context_t;

typedef struct { volatile uint32_t lock, owner, depth; } /* kernel lock */
typedef struct { volatile uint32_t lock; }              /* ticket lock */
```

plus the configuration macros in section 5.1.

### 3.2 Declarations — `cmsis-plus/rtos/port/os-decls.h`

The C++ half: the `port::stack`, `port::interrupts` and `port::scheduler` namespaces, and the four objects a port must **define**:

```
extern volatile state_t  lock_state[OS_NCPU];
extern smp_klock_t       _smp_klock;
extern smp_tlock_t       _smp_tlock;
extern volatile unsigned _port_ctx_pending[OS_NCPU];
```

This file contains no machine instructions, and all three ARM ports had it byte-identical but for one missing `volatile`. It therefore lives **once**, in the kernel repository at `port/smp-common/`, and a port opts in:

```
target_link_libraries (my-port INTERFACE micro-os-plus::port-smp-decls)
```

It is on no default include path. A port with different needs writes its own `os-decls.h` in its own `include/` and simply does not link that target, so the two can never be confused.

### 3.3 Inline operations — `cmsis-plus/rtos/port/os-inlines.h`

Per ISA. The kernel calls, and the port supplies:

| Function                                                | AArch32                                     | AArch64                                 |
|---------------------------------------------------------|---------------------------------------------|-----------------------------------------|
| <code>port_cpu_id_inline()</code>                       | <code>mrc p15, 0, Rd, c0, c0, 5</code>      | <code>mrs Xd, mpidr_el1</code>          |
| <code>interrupts::critical_section::enter/exit</code>   | <code>cpsid i</code> / <code>cpsie i</code> | <code>msr daifset/daifclr, #2</code>    |
| <code>interrupts::uncritical_section::enter/exit</code> | as above, inverted                          | as above, inverted                      |
| <code>scheduler::lock/unlock/locked</code>              | recursive kernel lock                       | recursive kernel lock                   |
| <code>_smp_klock_raw_acquire/release</code>             | <code>ldrex</code> / <code>strex</code>     | <code>ldaxr</code> / <code>stlxr</code> |

### 3.4 Functions with C linkage

```

unsigned port_cpu_id (void);          /* current core index, 0..OS_NCPU-1 */
void    port_sys_init (void);         /* per-core tick + IRQ controller */
void    port_ctx_switch (void);       /* context-switch entry */
void    port_smp_secondary_start (void); /* secondary core entry point */
void    smp_install_boot_threads (void); /* supplied by the application */

```

`smp_install_boot_threads()` is the one the *application* owes the port: it creates an idle thread per secondary CPU and registers each in `os::rtos::scheduler::os_idle_thread_core[]` before the cores are released. Most of a board's tests get it from that board's `test/src/test-smp-boot.cpp`; a test carried over whole from the predecessor brings its own, and says so through `BOARD_TEST_SELF_CONTAINED` in the board's `test/tests.cmake`.

### 3.5 Bring-up order

```

reset -> startup.S          core 0 only; others park
      -> os_main            application
      -> interrupts::uncritical_section::enter ()  unmask IRQs
      -> smp_install_boot_threads ()              idle thread per core
      -> smp::start_secondary_cores ()            release cores 1..N-1
      -> scheduler::start ()

```

Each secondary enters `port_smp_secondary_start()`, takes the kernel lock, initialises its tick through `port_sys_init()`, and schedules its idle thread.

## 4. What makes the test applications ISA-neutral

One copy of each test is compiled by every port that can run it. The disagreements between the old 32b and 64b copies resolved like this:

| Was                                                                               | Now                                                  | Note                                                             |
|-----------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------|
| <code>dsb</code> / <code>dmb</code> vs <code>dsb sy</code> / <code>dmb ish</code> | <code>dsb sy</code> , <code>dmb ish</code>           | not a difference: the explicit forms assemble on ARMv7-A as well |
| <code>cpsie i</code> vs <code>msr daifclr, #2</code>                              | <code>interrupts::uncritical_section::enter()</code> | a kernel API every port implements                               |
| <code>mrrc p15,...,c14</code> vs <code>mrs cntpct_el0</code>                      | <code>timer_arm::get_count()</code>                  | the port already had it                                          |
| "(AArch64)" in a banner                                                           | <code>PORT_BANNER_ISA</code>                         | supplied by the port's <code>uart.hpp</code>                     |

The rule: **an application names no register and no instruction that is not portable across the ports that can run it.** What it needs from a port it takes through the port's headers.

## 5. Preprocessor defines, per folder

### 5.1 `micro-os-plus-iii-aarch32` / `-aarch64` — defined by the port header

Set in `include/cmsis-plus/rtos/port/os-c-decls.h`; the build may override the first two.

| Macro                                                          | Value                                                 | Meaning                                                              |
|----------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------|
| <code>OS_NCPU</code>                                           | 4 ( <code>#ifndef</code> )                            | core count; the build chooses                                        |
| <code>OS_USE_SMP_SCHEDULER</code>                              | 1 when <code>OS_NCPU &gt; 1</code>                    | selects the SMP scheduler over the scalar one                        |
| <code>SMP_NO_OWNER</code>                                      | <code>0xFFFFFFFFu</code>                              | "kernel lock unowned" sentinel                                       |
| <code>OS_SMP_IPI_SGI</code>                                    | 0                                                     | cross-core reschedule signal; BCM2837 has no GIC, so local mailbox 0 |
| <code>OS_INTEGER_RTOS_STACK_FILL_MAGI</code><br><code>C</code> | <code>0xEFBEADDE</code> / <code>...EFBEADDEULL</code> | stack paint; element width                                           |
| <code>OS_HAS_INTERRUPTS_STACK</code>                           | defined                                               | the port provides a separate IRQ stack                               |

### 5.2 Set by the build for every application

`uos_add_app()` in `cmake/uos-app.cmake` :

| Macro                               | Source                                            |
|-------------------------------------|---------------------------------------------------|
| <code>OS_NCPU=&lt;n&gt;</code>      | <code>NCPU</code> argument                        |
| <code>OS_USE_SMP_SCHEDULER=1</code> | added automatically when <code>NCPU &gt; 1</code> |

The architecture project's `test/CMakeLists.txt` :

| Macro                        | Where                            | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>TRACE</code>           | both                             | enable the trace channel                                                                                                                                                                                                                                                                                                                                                                                                              |
| <code>__ARM_EABI__</code>    | both                             | ARM EABI                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <code>__ARM_ARCH_7A__</code> | aarch32 only                     | ARMv7-A                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>SEMIHOST</code>        | both                             | mirror the console to the semihosting channel and exit through it once a <b>RESULT</b> is printed. <b>On by default</b> , as it was in the Makefiles, so no build is silently non-semihosting                                                                                                                                                                                                                                         |
| <code>BOARD_RPI3B</code>     | <code>-DBOARD=rpi3b</code>       | Pi 3 B instead of Pi Zero 2 W: other linker script, other banner                                                                                                                                                                                                                                                                                                                                                                      |
| <code>HW_BUILD</code>        | <code>hwd</code> variant         | SD tests use the existing FAT32 boot partition through FatFs instead of formatting a blank card                                                                                                                                                                                                                                                                                                                                       |
| <code>DEBUG_BOOT</code>      | <code>-DUOS_DEBUG_BOOT=ON</code> | early-boot asm markers from <code>startup.S</code> , for bring-up under OpenOCD. <b>Off by default</b> , as the sources assume: each marker is a semihosting trap, and under a JTAG probe a trap costs real time                                                                                                                                                                                                                      |
| <code>LED_PIN</code>         | <code>BOARD=rpi-zero-2w</code>   | the GPIO driven as the user LED, 29 by default — the Zero 2 W's onboard green ACT LED. <code>led.hpp</code> 's own fallback is GPIO 16, header pin 36, which blinks nothing on a bare board. Which pin it is depends on the <b>board</b> , so it is set here for every test rather than per application; <code>BOARD=rpi3b</code> does not use it at all, because there the ACT LED is VideoCore expander GPIO 130 behind the mailbox |

### 5.3 Per-application, from the board's `test/tests.cmake`

Only `usb_test` has knobs:

| Macro                                            | Default | Meaning                                                                                                                                                                                                                                         |
|--------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>LED_BLINKS</code>                          | 3       | blinks per burst                                                                                                                                                                                                                                |
| <code>LED_ON_MS</code> / <code>LED_OFF_MS</code> | 40 / 40 | half-cycles; the tick is 1 ms, so each is clamped to at least that                                                                                                                                                                              |
| <code>LED_GAP_MS</code>                          | 300     | dark gap between bursts                                                                                                                                                                                                                         |
| <code>USB_FORCE_FS</code>                        | defined | force full speed. It also sets the driver's <code>kMpsHs</code> to 64, which is what bounds a bulk transfer: <code>D{I,0}EPTSIZ.PKTCNT</code> is 10 bits on this core, so 1023 packets — 65472 bytes at full speed against 523776 at high speed |

Consumed but not set by default:

| Macro                              | Meaning                                                                                                                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RUN_MS</code>                | wall-clock run length for <code>smp-num-test</code> , <code>smp-pipeline-test</code> , <code>smp-pro-cons-test</code> . The sources carry their own defaults; set it for short QEMU smoke runs |
| <code>TEST_IDLE_STACK_WORDS</code> | stack words per secondary idle thread, default 512                                                                                                                                             |

## 5.4 `micro-os-plus-iii-smp` — the kernel

The kernel reads a large set of optional `OS_*` switches (`OS_TRACE_RTOS_*`, `OS_INCLUDE_RTOS_STATISTICS_*`, `OS_USE_RTOS_PORT_*`, ...). **None is required.** The ARM builds set none of them; every one defaults to off.

Two are structural:

| Macro                               | Meaning                                                                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>OS_USE_OS_APP_CONFIG_H</code> | every kernel TU includes <code>&lt;cmsis-plus/os-app-config.h&gt;</code> from the port. The ARM ports keep that file empty and do not define this |
| <code>OS_USE_SMP_SCHEDULER</code>   | selects the SMP scheduler. Comes from the port header or the build, never hand-written in a source                                                |

## 5.5 `micro-os-plus-iii-devices`

FatFs is configured entirely through `fatfs/ffconf.h` (`FF_*`, ~40 macros); nothing needs to be passed on the command line. The only build-visible macro is `USB_FORCE_FS`, which `usb_test` sets.

Device sources dispatch on `__aarch64__` inline where an instruction genuinely differs — three places in total, for the cache maintenance and the core-id read. No project-specific macro selects an ISA.



## 6. Verifying a port

---

```
tools/verify-kernel-compiles.sh <port-include-dir> [compiler]
```

Compiles every source the kernel declares against that port's headers. It is the only meaningful standalone check: the kernel can never compile on its own, because `include/cmsis-plus/rtos/os-decls.h` includes `<cmsis-plus/rtos/port/os-decls.h>`, which only a port supplies.

Current result: **61 declared, 61 compiled.**